

Demake de Journey of the Prairie King

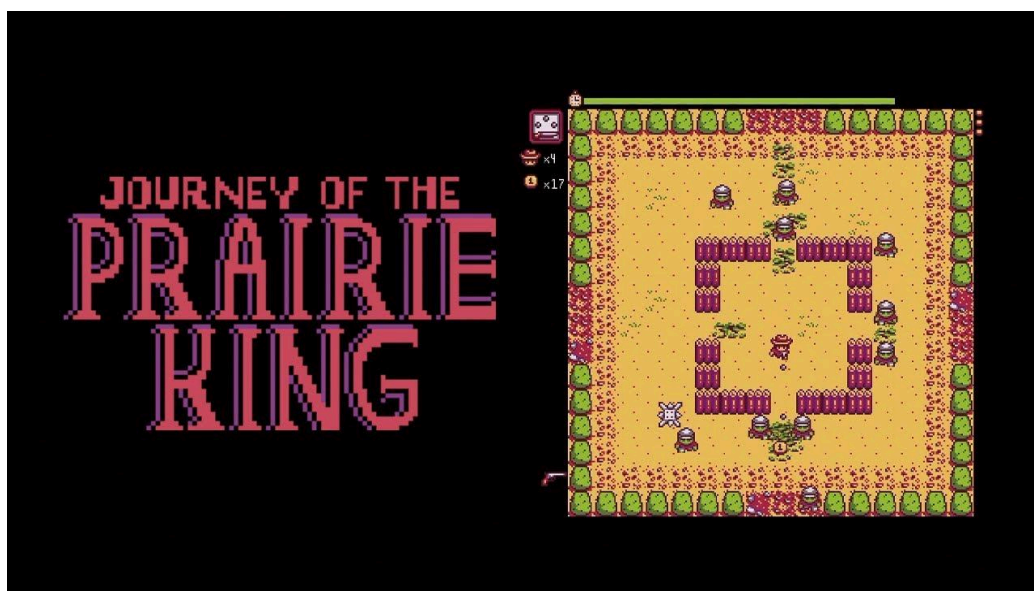
João Pedro-261032445 Arthur Vitor-261027267 Simão Silva-261042325

University of Brasília, Dept. of Computer Science, Brazil

Resumo

Esse artigo científico foi feito para descrever o desenvolvimento e o resultado do projeto que foi criado pelos alunos com o intuito de recriar o “Journey of the Prairie King” em Assembly RISC-V, utilizando o RARS como ferramenta de desenvolvimento.

1. Introdução



Em “Stardew Valley”, videogame lançado por ConcernedApe em 2016, o jogador tem acesso ao Stardrop Saloon, onde estão algumas máquinas de arcade que o possibilitam a entrar em mini-games, sendo um desses o “Journey of the Prairie King”. O mini-jogo original consiste no herói armado precisando sobreviver por uma certa quantidade de tempo em arenas cheias de obstáculos enquanto ele está sendo atacado por várias hordas de inimigos(cada um com sua habilidade única), tendo acesso à power-ups(como, os que aumentam a taxa de tiros ou a forma como os tiros se propagam pelo mapa) para facilitar a sua jornada.

Além disso, também há um sistema de compras para adquirir itens que ajudam no progresso do jogador, e existe a necessidade de se confrontar um boss antes de se prosseguir para a próxima fase. Dessa forma, essa gameplay interessante, somado aos gráficos e trilha sonora simples mas geniais, tornou esse mini-jogo muito memorável e amado pelos fãs.

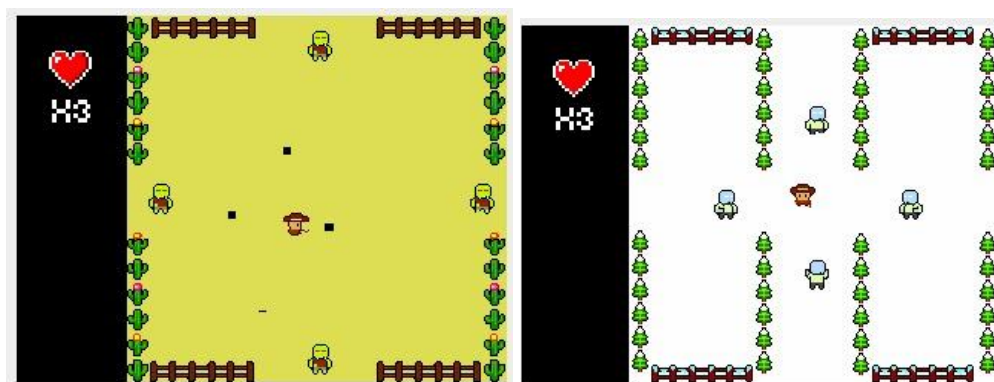
2. Metodologia

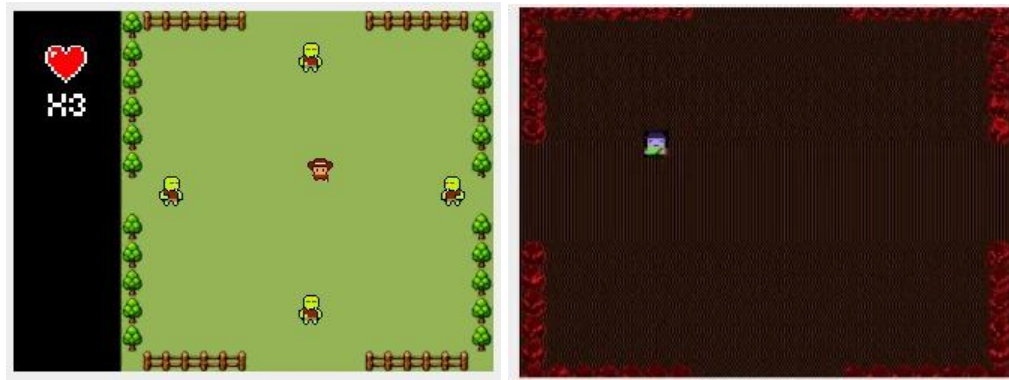


2.1 Layouts

Para a criação dos mapas do jogo, utilizamos como referência vídeos produzidos por veteranos da disciplina que demonstravam o processo de conversão de imagens para o formato utilizado pelo RARS em Assembly RISC-V. Os cenários são representados por uma matriz de 320×240 posições, em que cada elemento corresponde à cor de um pixel exibido na tela. Após a conversão, a imagem é armazenada em um vetor de bytes, permitindo que o cenário seja carregado e desenhado corretamente durante a execução do jogo.

No total, foram feitos 3 mapas diferentes (deserto, neve e gramado) e, por curiosidade, também acidentalmente foi feito um mapa “vulcânico” que se originou de um bug na conversão do png para matriz, mas que infelizmente não chegou na versão final.





2.2 Animações e movimentações

Para a movimentação foi utilizada a chamada do teclado para definir a direção por meio das teclas W, A, S, D. A cada comando a posição do player é atualizada em 8 pixels na direção correspondente, desde que não haja colisão com algum obstáculo do mapa. Antes da atualização da posição do player é feito uma verificação no mapa de colisão da respectiva fase impedindo que o jogador atravesse paredes ou áreas bloqueadas. Além disso, o estado do personagem é alterado conforme a direção do movimento, permitindo a exibição do sprite correspondente.

```
#####
#  Movimentacao do player  #
#####

mover_cima:

    la t0, PLAYER_POS      #Pegando o endereco da posicao do jogador
    lh t1, 2(t0)            #ler da memoria o offset 2 = y
    addi t1, t1, -8         #Subtrai -8 pixels

    #checar as colisoes
    li a3, 17              #largura e altura
    li a4, 17
    mv a2, t1              #movo o y
    lh a1, 0(t0)           #leio x

    addi sp, sp, -4         #salvo o ra e checo as colisoes
    sw ra, 0(sp)
    call colisao
    lw ra, 0(sp)
    addi sp, sp, 4

    li t6, 1               #se retorna = 1 pula para tecla_fim
    beq a0, t6, tecla_fim
```

2.3 Ataques

O ataque do jogador é realizado pelas teclas I, J, K, L, que disparam projéteis nas direções cima, esquerda, baixo e direita. O tiro é criado na posição atual do personagem e segue a direção escolhida pelo jogador. Caso encontre um obstáculo ou atinja um inimigo, o tiro é desativado e removido da tela. Por fim, um efeito sonoro é reproduzido sempre que o player atira.

2.4 Colisões

Para o sistema de colisão foi feito um mapa de colisão gerado a partir do sprite do cenário. Esse mapa consiste em uma matriz de 320x240 na qual cada elemento representa um pixel do cenário. Nessa matriz são utilizados os valores

0 e 1, em que 0 indica uma área livre para movimentação e 1 representa um obstáculo. Sempre que o jogador, tiro ou inimigo tenta se mover, o programa consulta a posição correspondente na matriz antes de atualizar. Caso o valor encontrado seja 1, o movimento é bloqueado, caso seja 0, a movimentação é permitida.

2.5 Inimigos e condição de derrota/vitória

Para os inimigos foi implementado um sistema que persegue automaticamente o jogador durante toda a partida. A cada frame, o jogo verifica quais inimigos estão ativos, compara suas posições com a posição do jogador e movimenta cada um na direção que reduz a distância entre eles, além de atualizar o sprite de acordo com a direção em que estão se movendo. Também foi desenvolvido uma condição de vitória de se o jogador matar em cada um dos 3 níveis 8 inimigos e uma de derrota caso o jogador for encostado pelo inimigo e perder todas suas vidas, o que faz ele entrar na tela de game over.

2.6 Música

O sistema de músicas foi implementado utilizando as chamadas MIDI disponíveis no RARS. Para facilitar a obtenção das notas das trilhas sonoras, foi utilizado um vídeo como referência, no qual era disponibilizado um script para o site HookTheory. Esse script retornava automaticamente a quantidade de notas, bem como os valores das notas e suas respectivas durações. Após a extração, essas informações foram armazenadas em vetores no programa e reproduzidas por meio das chamadas MIDI durante o menu, a vitória e o game over.

2.7 HUD

No HUD, que é uma barra lateral esquerda preta, é exibido a quantidade de vidas restantes por meio de um ícone de coração acompanhado de um indicador numérico. O sistema atualiza essas informações sempre que o jogador sofre dano, até chegar na última vida, em que ocorre o game over.

```
#####
#      HUD      #
#####

la a0, sprite_coracao_dados #Carrego o endereco do coracao
li a1, 20
li a2, 20
li a3, 30
li a4, 30
li a5, 0

addi sp, sp, -4    #salva o ra e imprime o cenario
sw ra, 0(sp)
call Print
lw ra, 0(sp)
addi sp, sp, 4

la t0, player_vida #carrego o endereco do player_vida
lw t1, 0(t0)        #achar a posicao da vida
addi t3, t1, -1
li t4, 12
mul t3, t3, t4

la t5, placar_vida #carregar o endereco e os dados do sprite da vida
add t5, t5, t3
```

3. Resultados Obtidos

Dentre os principais resultados do projeto final, vale destacar: o avanço de fazer as coisas se moverem, em que o grupo conseguiu criar um código que faz um bloco ou personagem andar para os lados, para cima e para baixo, respondendo na hora na tela de testes. A organização da memória do sistema para o programa não travar, que era uma parte bem complicada em Assembly, mas o código ficou limpo e rodou de forma leve. Os comandos criados funcionaram sem atrasos, ou seja, quando o sistema recebia uma ordem, a ação acontecia direto na tela, mostrando que a lógica deu certo. E, por fim, toda a parte de explicações e o passo a passo de como usar o sistema foi terminada, o que deixou fácil para qualquer pessoa entender como o código foi montado e como fazer ele funcionar.

4. Conclusão

Por meio dos conhecimentos adquiridos durante as aulas de Introdução aos Sistemas Computacionais e do uso da Arquitetura de instruções reduzidas desenvolvida na Universidade da Califórnia, foi possível superar os desafios dessa proposta e entregar um resultado que atende às especificações propostas e, mesmo com certas simplificações das mecânicas de seu antecessor, ainda faz jus à grandeza do jogo original.

Assim, apesar das dificuldades enfrentadas ao longo do desenvolvimento, conseguimos concluir o projeto. Além do aprendizado adquirido durante o processo, o desenvolvimento foi uma experiência bastante satisfatória, pois acompanhar a evolução do projeto e ver todas as funcionalidades funcionando nos trouxe bastante orgulho e satisfação.

5. Referências Bibliográficas

[Convertendo imagens pra usar no RARS. Youtube, 2021.](#)

[Como importar músicas do Hooktheory para tocar no RARS. Youtube, 2021.](#)

[Renderização dinâmica no Bitmap Display. Youtube, 2021.](#)

[STARDEW Valley Wiki. Journey of the Prairie King. 2026.](#)

Aulas do professor Marcus Vinicius Lamar